XSS

CS 161 Spring 2026 - Lecture 19

Last Time: Cookies

- Cookie: a piece of data used to maintain state across multiple requests
  - Set by the browser or server
  - Stored by the browser
  - Attributes: Name, value, domain, path, secure, HttpOnly, expires
- Cookie policy
  - Server with domain X can set a cookie with domain attribute Y if the domain attribute is a **domain suffix** of the server's domain, and the domain attribute Y is not a top-level domain (TLD)
  - The browser attaches a cookie on a request if the domain attribute is a **domain suffix** of the server's domain, and the path attribute is a **prefix** of the server's path
  - Cookie domain: example.com and cookie path: /some/path will be included in request to https://foo/example/com/some/path/index.html

Last Time: Session Authentication

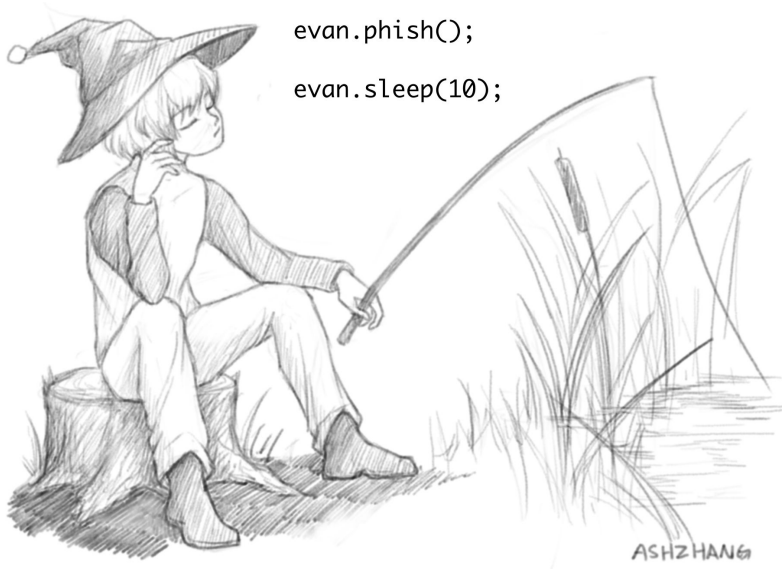
- Session authentication
  - Use cookies to associate requests with an authenticated user
  - First request: Enter username and password, receive session token (as a cookie)
  - Future requests: Browser automatically attaches the session token cookie
- Session tokens
  - If an attacker steals your session token, they can log in as you
  - Should be randomly and securely generated by the server
  - The browser should not send tokens to the wrong place

Last Time: CSRF

- Cross-site request forgery (CSRF or XSRF): An attack that exploits cookie-based authentication to perform an action as the victim
  - User authenticates to the server
    - User receives a cookie with a valid session token
  - Attacker tricks the victim into making a malicious request to the server
  - The server accepts the malicious request from the victim
    - Recall: The cookie is automatically attached in the request
- Attacker must trick the victim into creating a request
  - GET request: click on a link
  - POST request: use JavaScript

Today: XSS

- XSS
  - Websites use untrusted content as control data
  - Stored XSS
  - Reflected XSS
  - Defense: HTML sanitization
  - Defense: Content Security Policy (CSP)
- UI attacks
  - Clickjacking
  - Phishing



Cross-Site Scripting (XSS)

Textbook Chapter 22

Top 25 Most Dangerous Software Weaknesses (2020)

Rank	ID	Name	Score
[1]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	46.82
[2]	CWE-787	Out-of-bounds Write	46.17
[3]	CWE-20	Improper Input Validation	33.47
[4]	CWE-125	Out-of-bounds Read	26.50
[5]	CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer	23.73
[6]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	20.69
[7]	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	19.16
[8]	CWE-416	Use After Free	18.87
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	17.29
[10]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	16.44
[11]	CWE-190	Integer Overflow or Wraparound	15.81
[12]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	13.67
[13]	CWE-476	NULL Pointer Dereference	8.35
[14]	CWE-287	Improper Authentication	8.17
[15]	CWE-434	Unrestricted Upload of File with Dangerous Type	7.38
[16]	CWE-732	Incorrect Permission Assignment for Critical Resource	6.95
[17]	CWE-94	Improper Control of Generation of Code ('Code Injection')	6.53

Review: Same-Origin Policy

- Two webpages with different origins should not be able to access each other's resources
  - Example: JavaScript on **`http://evil.com`** cannot access the information on **`http://bank.com`**

Review: JavaScript

- **JavaScript**: A programming language for running code in the web browser
- JavaScript is **client-side**
  - Code sent by the server as part of the response
  - Runs in the browser, not the web server!
- Used to manipulate web pages (HTML and CSS)
  - Makes modern websites interactive
  - JavaScript can be directly embedded in HTML with `<script>` tags
- Most modern webpages involve JavaScript
  - JavaScript is supported by all modern web browsers
- You don’t need to know JavaScript syntax
  - However, knowing common attack functions helps

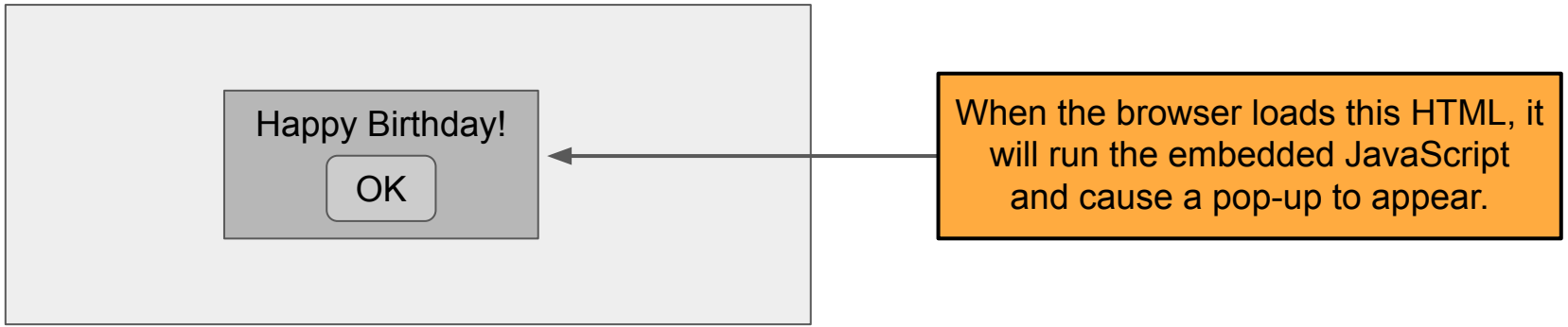
Review: JavaScript

- JavaScript can create a pop-up message

HTML (with embedded JavaScript)

```
<script>alert("Happy Birthday!")</script>
```

Webpage



A Go HTTP Handler

Handler

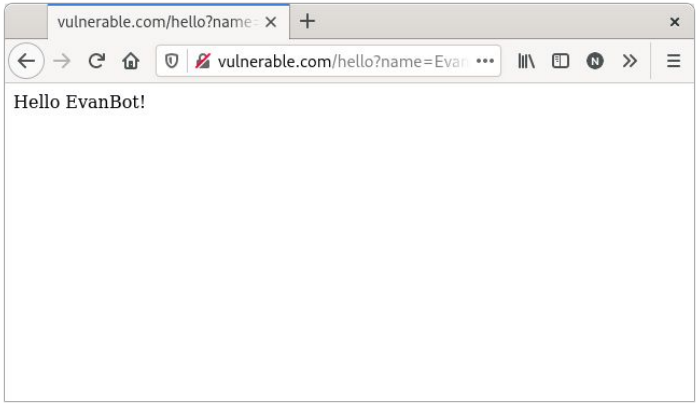
```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=EvanBot
```

Response

```
<html><body>Hello EvanBot!</body></html>
```



A Go HTTP Handler

Handler

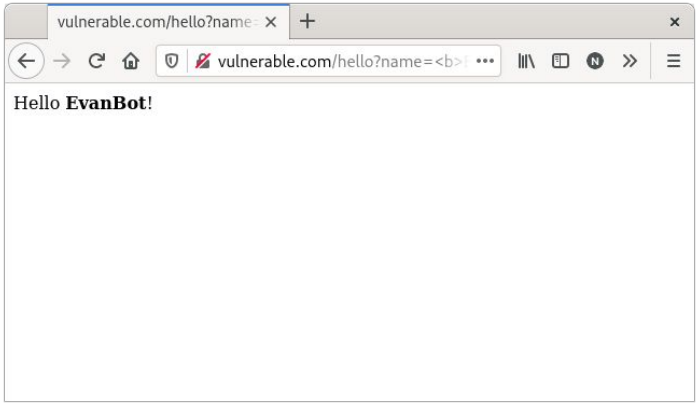
```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=<b>EvanBot</b>
```

Response

```
<html><body>Hello <b>EvanBot</b>!</body></html>
```



A Go HTTP Handler

Problem: This input represents control data (HTML), not just text!

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", name)
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello <script>alert(1)</script>!</body></html>
```



A Go HTTP Handler

Not just `%s`: It can happen with any string manipulation

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    content := "<html><body>Hello "+name+"!</body></html>"
    fmt.Fprint(w, content)
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello <script>alert(1)</script>!</body></html>
```



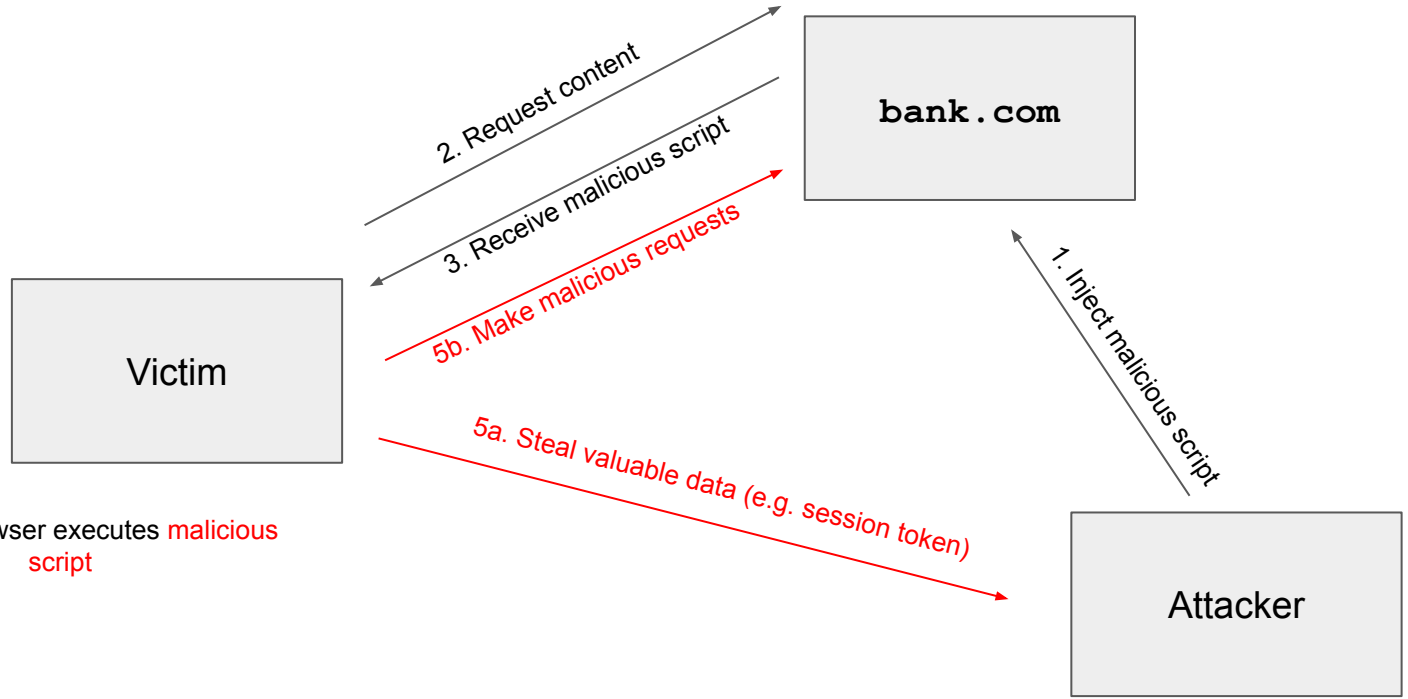
Cross-Site Scripting (XSS)

- Idea: The attacker adds malicious JavaScript to a legitimate website
  - The legitimate website will send the attacker's JavaScript to browsers
  - The attacker's JavaScript will run with the origin of the legitimate website
  - Now the attacker's JavaScript can access information on the legitimate website!
- **Cross-site scripting** (**XSS**): Injecting JavaScript into websites that are viewed by other users
  - Cross-site scripting subverts the same-origin policy
- Two main types of XSS
  - Stored XSS
  - Reflected XSS

Stored XSS

- **Stored XSS** (**persistent XSS**): The attacker's JavaScript is **stored** on the legitimate server and sent to browsers
- Classic example: Facebook pages
  - An attacker puts some JavaScript on their Facebook page
  - Anybody who loads the attacker's page will see JavaScript (with the origin of Facebook)
- Remember: Stored XSS is a server-side vulnerability!

Stored XSS

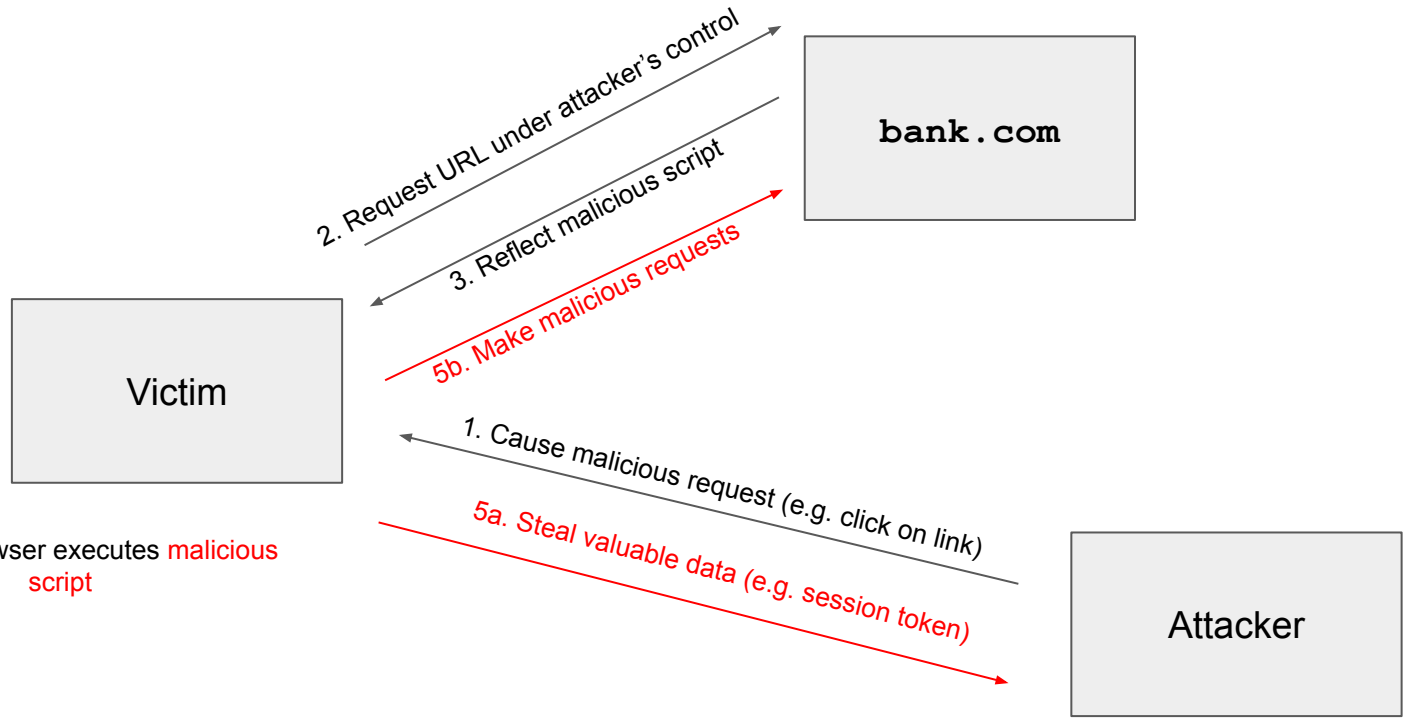


4. Victim browser executes malicious script

Reflected XSS

- **Reflected XSS**: The attacker causes the victim to input JavaScript into a request, and the content is **reflected** (copied) in the response from the server
- Classic example: Search
  - If you make a request to `http://google.com/search?q=evanbot`, the response will say "10,000 results for evanbot"
  - If you make a request to `http://google.com/search?q=<script>alert(1)</script>`, the response will say "10,000 results for `<script>alert(1)</script>`"
- Reflected XSS requires the victim to make a request with injected JavaScript

Reflected XSS



4. Victim browser executes malicious script

Reflected XSS: Making a Request

- How do we force the victim to make a request to the legitimate website with injected JavaScript?
  - Trick the victim into visiting the attacker's website, and include an embedded iframe that makes the request
    - Can make the iframe very small (1 pixel x 1 pixel), so the victim doesn't notice it:

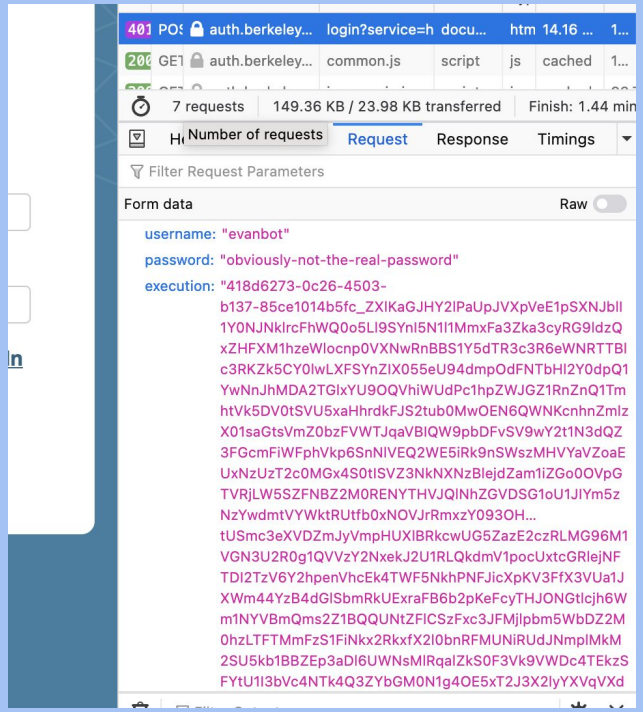
      ```
      <iframe height=1 width=1
      src="http://google.com/search?q=<script>alert(1)</script>">
      ```
  - Trick the victim into clicking a link (e.g. posting on social media, sending a text, etc.)
  - Trick the victim into visiting the attacker's website, which redirects to the reflected XSS link
  - … and many more!

Reflected XSS is not CSRF

- Reflected XSS and CSRF both require the victim to make a request to a link
  - XSS: An HTTP response contains maliciously inserted JavaScript, executed on the client side
  - CSRF: A malicious HTTP request is made (containing the user's cookies), executing an effect on the server side

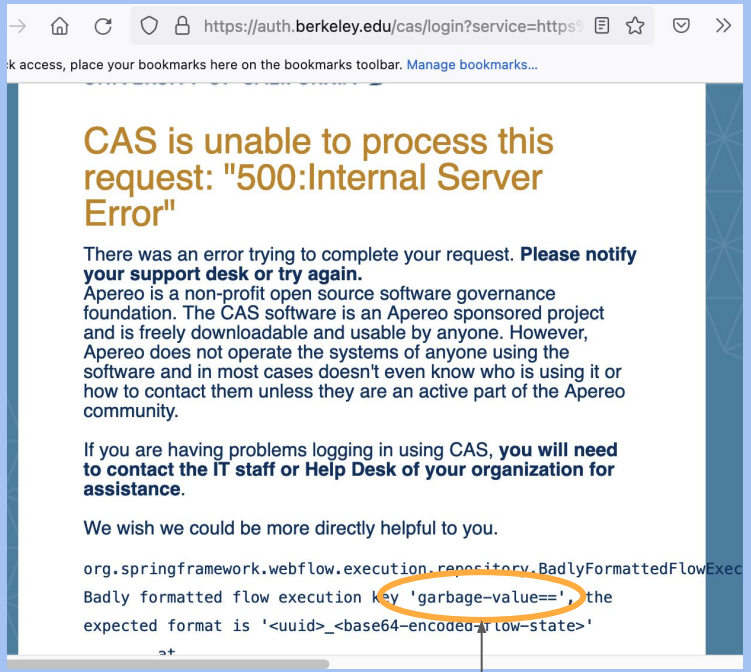
XSS in the Wild… On CalNet?!

- In 2021, 61C student Rohan Mathur was exploring the CalNet login page
- Fields submitted when logging in:
  - **`username`**: What user am I logging in as?
  - **`password`**: What's the user's password?
  - **`execution`**: Server-side state (like a CSRF token with extra data)



XSS in the Wild… On CalNet?!

- The server expects **`execution`** to be in a specific format that contains its server-side state
  - What if you muck with **`execution`**?
- The “corrupted” execution key is placed into the error message in the HTML to aid debugging
  - … and CalNet at the time did not escape the execution key



Constructing an Attack on CalNet

Attack: Force a POST request to CalNet!

```
<html>
  <head>
    <script>
      // When the malicious page finishes loading, automatically submit the form!
      document.addEventListener('DOMContentLoaded', () => {
        document.getElementById('form').submit();
      });
    </script>
  </head>
  <body>
    <!-- Malicious form containing our malicious execution data. -->
    <form id="form" action="https://auth.berkeley.edu/cas/login" method="POST">
      <input name="username" type="text" value="evanbot" />
      <input name="password" type="text" value="obviously-not-the-real-password" />
      <input name="execution" type="text" value="<script>alert('XSS!')</script>" />
    </form>
  </body>
</html>
```

So What Happened?

- CalNet also accepts **`execution`** parameters over URL query parameters
  - A link like **`https://auth.berkeley.edu/cas/login?execution=<script>alert('XSS!')</script>`** would result in the same attack
- It would only fire if you clicked the button to show the error
- … But if clicked, the injected JavaScript could
  - Present a fake login prompt
  - Steal your CalNet password
  - Log you in as the attacker's account
  - Steal your authentication session token
- Root cause: A >5 year old sample page modified by CalNet
  - CalNet uses software from Apereo, which comes with sample pages for logging in
  - Apereo fixed that bug many years before, but sample pages don't get included in bug fixes!

XSS Defenses

- Defense: **HTML sanitization**
  - Idea: Certain characters are special, so server should remove those characters or encode them in a way that will render them harmless
- Escaping: e.g., replace < with `<` with " with `"`
- Rely on trusted libraries to do this for you

```
<html>
<body>
Hello &lt;script&gt;alert(1)&lt;/script&gt;!
</body>
</html>
```

XSS Defenses: Escaping

Handler

```
func handleSayHello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query()["name"][0]
    fmt.Fprintf(w, "<html><body>Hello %s!</body></html>", html.EscapeString(name))
}
```

URL

```
https://vulnerable.com/hello?name=<script>alert(1)</script>
```

Response

```
<html><body>Hello &lt;script&gt;alert(1)&lt;/script&gt;!</body></html>
```

XSS Defenses: Escaping

- If a programmer has to take an extra action for every usage…
  - They are eventually going to forget (e.g., CalNet)
  - Consider human factors!
- Nowadays, escaping is generally achieved through **templating**
  - HTML templates are essentially their own language, where you declare what data goes where
  - The templating engine handles all the escaping internally, automatically

```
<html>
<body>
Hello {{.name}}!
</body>
</html>
```

Example: Golang HTML template

XSS Defenses: CSP

- Defense: **Content Security Policy** (**CSP**)
  - Idea: Instruct the browser to only use resources loaded from specific places
  - Uses additional headers to specify the policy
- Standard approach:
  - Disallow all inline scripts (JavaScript code directly in HTML), which prevents inline XSS
    - Example: Disallow `<script>alert(1)</script>`
  - Only allow scripts from specified domains, which prevents XSS from linking to external scripts
    - Example: Disallow `<script src="https://cs161.org/hack.js">`
- Also works with other content (e.g. iframes, images, etc.)
- Relies on the browser to enforce security, so more of a mitigation for defense-in-depth